

Resumo das Cheat Sheets

Claude IA, Claude Code e Claude Cowork

Oito colinhas de referência rápida — de anatomia de prompt a harness de LLM —
condensadas neste boletim, cada uma com seu PDF completo na pasta do módulo.

- Claude Cowork
- Engenharia de Prompt
- Claude Code
- Atalhos e Erros Comuns
- Harness de LLM
- Mapa de Decisão e Glossário
- Claude AI
- Agent Skills

O que este módulo cobre

Este documento reúne, em formato executivo, as oito colinhas de referência rápida produzidas neste módulo — cada uma já com um PDF próprio salvo em sua respectiva subpasta. São guias independentes entre si, cada um cobrindo uma ferramenta ou tema do ecossistema Claude (Claude AI, Claude Code, Claude Cowork, prompt engineering, harness de LLM, atalhos e o mapa/glossário que conecta tudo). Aqui você encontra o essencial de cada uma lado a lado.

01

Claude Cowork

automações direto do desktop — O Claude Desktop deixa de ser chat e vira operador de ferramentas: lê Drive, atualiza Linear, escreve no Notion, consulta Postgres — tudo via MCP, com Projects dando contexto fixo e Skills automatizando rotinas.

02

Engenharia de Prompt

como pedir muda tudo — Prompt fraco gera resposta genérica; prompt forte tem papel, contexto, tarefa, formato, restrições e exemplo. Especificidade vence verbosidade — saber quais dos seis blocos omitir é o que separa o iniciante do operador.

03

Claude Code

pra devs que querem ir além — O Claude Code lê o projeto inteiro, roda comandos, edita código e abre PRs direto no repo — contexto dado via CLAUDE.md, estendido com skills, hooks e MCP, operando como um par júnior que nunca dorme.

04

Atalhos e Erros Comuns

atalhos + armadilhas do ecossistema — Colinha “parte 2”: reúne lado a lado os atalhos de Claude AI, Claude Code e Claude Cowork, mais os erros mais comuns de cada ferramenta e os que aparecem só quando elas são usadas em conjunto.

05

Harness de LLM

o motor precisa de chassi — Modelo é o motor, harness é o carro. Harness é a camada de software que coloca um LLM num loop agente — orquestrando ferramentas, contexto, sistema de arquivos e permissões — comparando Claude Code/Cowork, OpenClaw, OpenCode e Codex.

06

Mapa de Decisão e Glossário

mapa + glossário do ecossistema — Três ferramentas, um ecossistema. Um mapa de decisão pra saber por onde começar, uma tabela de onde cada recurso (Projects, MCP, Skills...) vive e cada uma, e um glossário unificado dos termos que aparecem nas três.

07

Claude AI

pra quem quer pensar melhor — Mais que um chatbot: o Claude lê documentos longos, escreve código, analisa dados e raciocina com nuance. Acessível via claude.ai, apps, API e integrações MCP — a colinha cobre escolha de modelo, técnicas de prompt e os recursos além do chat simples.

08

Agent Skills

ensine uma vez, use sempre — Skill é uma pasta com um SKILL.md que empacota instruções, scripts e recursos pra o Claude executar tarefas específicas de forma consistente — carregada automaticamente quando o pedido casa com a description.

01 Claude Cowork

automações direto do desktop

O Claude Desktop deixa de ser chat e vira operador de ferramentas: lê Drive, atualiza Linear, escreve no Notion, consulta Postgres — tudo via MCP, com Projects dando contexto fixo e Skills automatizando rotinas.

- MCP é o motor: conectores (filesystem, GitHub, Linear/Jira, Slack, Drive/Notion, Postgres) instalados em `claude_desktop_config.json` — comece com 2-3 relevantes e cresça.
- Project = pasta com instruções fixas sempre disponíveis; Skill = rotina ensinada uma vez (pasta + SKILL.md), reusada pra sempre.
- Oito workflows prontos: Daily Standup, Inbox Triage, Weekly Report, Doc Sync, Customer Recap, Bulk Drafts, Migration Plan, Meeting Notes.
- Armadilhas do modo Cowork: MCP demais de uma vez, “Allow always” sem ler, Project sem instruções, workflow sem dono.
- Quando não usar: tarefas triviais (use o web), automações que precisam rodar sem você (prefira Zapier/n8n), dados ultra-sensíveis sem revisão.

02 Engenharia de Prompt

como pedir muda tudo

Prompt fraco gera resposta genérica; prompt forte tem papel, contexto, tarefa, formato, restrições e exemplo. Especificidade vence verbosidade — saber quais dos seis blocos omitir é o que separa o iniciante do operador.

- Anatomia em 6 blocos: Papel, Contexto, Tarefa, Formato, Restrições, Exemplo.
- Cinco técnicas centrais: Chain-of-Thought, Few-Shot, prompts negativos, XML pra contexto complexo e iteração na mesma conversa.
- Quatro templates prontos pra colar: análise-de-dados, revisão-de-código, tomada-de-decisão, resumo-crítico.
- Erros recorrentes: pedir tudo de uma vez, contexto implícito, opinião sem critério definido, não pedir a fonte, sessão eterna trocando de assunto.
- Guia de escolha: Chain-of-Thought para problema complexo, Few-Shot para tom específico, XML para muitos inputs, negativos + persona para resposta genérica.

03 Claude Code

pra devs que querem ir além

O Claude Code lê o projeto inteiro, roda comandos, edita código e abre PRs direto no repo — contexto dado via CLAUDE.md, estendido com skills, hooks e MCP, operando como um par júnior que nunca dorme.

- Estrutura do projeto: CLAUDE.md na raiz + .claude/ (settings.json, commands/, agents/, skills/, hooks/); hierarquia global → projeto → subpasta, o mais específico vence.
- CLAUDE.md eficaz: imperativo vence descritivo, liste o que NÃO fazer, mantenha abaixo de 200 linhas, comece sempre com /init.
- Comandos essenciais: /init, /clear, /compact, /model, /cost, /review, /pr, /agents, /mcp, /resume.
- Atalhos de terminal: Shift+Tab (auto-accept on/off), Esc (interrompe), Esc Esc (edita última msg), Ctrl+R (busca histórico), ! (roda shell), # (atualiza memória).
- Cobrança via plano Pro/Max ou pay-as-you-go pela API — use /cost para acompanhar o gasto da sessão.

04 Atalhos e Erros Comuns

atalhos + armadilhas do ecossistema

Colinha “parte 2”: reúne lado a lado os atalhos de Claude AI, Claude Code e Claude Cowork, mais os erros mais comuns de cada ferramenta e os que aparecem só quando elas são usadas em conjunto.

- Atalhos por ferramenta: Claude AI (Shift+Enter, ↑, Ctrl/Cmd+K, @, /), Claude Code (/init.../help, Shift+Tab, Esc), Cowork (Cmd+Shift+., @projeto, Allow once, Stop).
- Erros no Claude AI: prompt vago, aceitar a 1ª resposta, não dar contexto, confiar em fatos sem checar, conversas eternas, usar Opus pra tudo.
- Erros no Claude Code: pular o /init, CLAUDE.md gigante, não rodar /review antes do PR, descrever em vez de mandar, deixar o Code decidir arquitetura.
- Erros no Cowork: MCP demais de uma vez, “Allow always” sem ler, Project sem Instructions, não revisar antes de executar, workflow sem dono.
- Erros que cruzam fronteiras: contexto solto entre ferramentas, ferramenta errada pra tarefa (pensar ≠ executar ≠ automatizar), não iterar, Skills desatualizadas.

05 Harness de LLM

o motor precisa de chassi

Modelo é o motor, harness é o carro. Harness é a camada de software que coloca um LLM num loop agente — orquestrando ferramentas, contexto, sistema de arquivos e permissões — comparando Claude Code/Cowork, OpenClaw, OpenCode e Codex.

- Anatomia em 5 peças: loop agente (plan→act→observe→reflect), tool layer, context manager, guard-rails, interface. “Loop > modelo”: loop excelente com modelo médio bate modelo top com loop ruim.
- Quatro harnesses comparados: Claude Code/Cowork (Anthropic, CLI+desktop, ecossistema integrado), OpenClaw (comunidade, BYOK, open-source, auditável), OpenCode (comunidade, TUI rica, multi-provider), Codex (OpenAI, CLI+Web/IDE, GPT nativo).
- Quatro modos de execução/autonomia: Read-only, Suggest, Auto-edit, Full-auto — bons harnesses deixam escolher o nível por tarefa.
- Combos poderosos: Harness+MCP, Harness+Skills, Harness+Subagent, e dois harnesses no mesmo fluxo (Cowork pro planejamento + Claude Code pra execução técnica).
- Checklist de adoção: definir a tarefa-âncora, escolher o harness, começar em modo suggest, habilitar MCP/plugins aos poucos, empacotar padrões em Skills, revisar mensalmente.

06 Mapa de Decisão e Glossário

mapa + glossário do ecossistema

Três ferramentas, um ecossistema. Um mapa de decisão pra saber por onde começar, uma tabela de onde cada recurso (Projects, MCP, Skills...) vive em cada uma, e um glossário unificado dos termos que aparecem nas três.

- Mapa de decisão: pensar/escrever → Claude AI; codar/commitar → Claude Code; automatizar/operar → Claude Cowork; não sabe ainda → comece no Claude AI e migre pra execução depois.
- Tabela “onde cada recurso vive”: Projects, MCP, Skills, Modelos, Artifacts e Web Search mudam de formato e poder entre as três ferramentas.
- Quatro combos entre ferramentas: Planejamento → Implementação, Pesquisa → Documento → Distribuição, Bug → Fix → Notificação, Dados → Análise → Ação.
- Glossário unificado: MCP, Project, Skill aparecem nas três; Artifact é só do Claude AI; CLAUDE.md, Hooks, Subagents e Slash commands são só do Claude Code; Instructions é o equivalente funcional em AI e Cowork.

07 Claude AI

pra quem quer pensar melhor

Mais que um chatbot: o Claude lê documentos longos, escreve código, analisa dados e raciocina com nuance. Acessível via claude.ai, apps, API e integrações MCP — a colinha cobre escolha de modelo, técnicas de prompt e os recursos além do chat simples.

- Três modelos, três usos: Opus (raciocínio profundo, projetos complexos), Sonnet (cavalo de batalha, 90% do dia a dia), Haiku (rápido, barato, volume) — comece no Sonnet, suba pra Opus quando travar.
- Técnicas que funcionam: few-shot, pensar passo a passo, XML pra estruturar, formato de saída definido, dizer o que NÃO fazer, iterar em vez de recomençar.
- Oito recursos além do chat: Projects, Artifacts, Files (lê PDFs/planilhas de fato), Web Search, Skills, MCP, Styles e Computer Use (ainda experimental).
- Erros recorrentes: prompt vago, aceitar a primeira resposta, não dar contexto, confiar em fatos sem checar, conversas eternas, usar Opus pra tudo o que é trivial.

08 Agent Skills

ensine uma vez, use sempre

Skill é uma pasta com um SKILL.md que empacota instruções, scripts e recursos pra o Claude executar tarefas específicas de forma consistente — carregada automaticamente quando o pedido casa com a description.

- Anatomia do SKILL.md: frontmatter YAML (name + description) é o gatilho de ativação — quanto mais precisa a description, mais certo o disparo; description vago não dispara nada.
- Progressive disclosure: só name + description entram no contexto no startup; o conteúdo completo só carrega quando a Skill é acionada — dá pra ter 50 Skills instaladas sem inflar cada conversa.
- Roda em 4 portas de entrada: Claude AI (Settings → Skills), Claude Code (.claude/skills/ ou /plugin install), Claude API (Skills API) e AWS/Foundry — mesmo padrão de pasta em todas.
- Skills vs CLAUDE.md vs Instructions: escopo de tarefa específica e carregamento sob demanda (Skill) contra projeto inteiro/toda sessão (CLAUDE.md) e workspace do Project/toda conversa (Instructions).
- Filtro de 4 perguntas antes de criar uma Skill: vai repetir 3+ vezes, tem passos específicos, precisa de resultado consistente entre sessões, envolve contexto organizacional.

Onde encontrar cada colinha

Este resumo é um ponto de entrada — cada colinha listada abaixo tem seu PDF completo, com todos os detalhes, exemplos e blocos de código, na respectiva subpasta deste módulo.

01	Claude Cowork	Cheat Sheet - Claude Cowork/Colinha do Claude Cowork.pdf
02	Engenharia de Prompt	Engenharia de Prompt/Colinha do Prompt 2.pdf
03	Claude Code	Cheat Sheet - Claude Code/Colinha do Claude Code.pdf
04	Atalhos e Erros Comuns	Atalhos e Erros Comuns/Atalhos e Erros Comuns.pdf
05	Harness de LLM	Harness de LLM - Claude, Codex, OpenCode, OpenClaw/Harness.pdf
06	Mapa de Decisão e Glossário	Mapa de Decisão e Glossário/Mapa de Decisão e Glossário.pdf
07	Claude AI	Cheat Sheet - Claude AI/Colinha do Claude AI.pdf
08	Agent Skills	Agent Skills/Colinha Agent Skill.pdf

Conteúdo extraído diretamente dos 8 PDFs originais de cada colinha. Este resumo consolida, em formato executivo, o material já produzido neste módulo da Formação Claude Code 2026.